



HTML: Semantic, Tables & Forms

Web Design and Programming

M.Sc. Tran Quang Toan

Faculty of Data Science and Artificial Intelligence



Session Outline

1. HTML Overview & Document Skeleton

2. Semantic HTML

3. Tables

4. Forms & Data Submission

5. Accessibility Basics



Learning Objectives

By the end of this session, you will be able to:

1. Build an HTML document skeleton and use basic content tags.
2. Explain why semantic HTML matters for accessibility & maintainability.
3. Build a valid HTML table for structured data.
4. Create a data-entry form; distinguish GET vs. POST.
5. Apply basic accessibility habits to your own pages.



How to Approach This Session

- This is the **only dedicated HTML session** – the goal is **enough to build your project screens**, not to memorize every tag.
- Focus on the **core 20% of tags that cover 80% of real use**; deeper detail is marked *Further Reading* for self-study.
- In the trio of web languages: **HTML = structure** (the skeleton). CSS styles it (Week 3), JavaScript brings it to life (Week 4).
- Tied to your project: by the end of today you will have an HTML skeleton ready to code your project's screens (Milestones 1-2).



Environment Check

- Open **VS Code**, install the **Live Server** extension.
- Create `index.html`, click “**Go Live**” – confirm a blank page loads in your browser.
- Stay ready: after each part, we code directly into this same page.



HTML Overview & Document Skeleton



What Is HTML?

- HTML is made of nested **elements** that form a **tree**.
- Each element = **opening tag** + **content** + **closing tag**: `<p>Hello there</p>`.
- Elements can carry **attributes**: `<a href="...">link</a>`.
- Some tags are **self-closing**, with no content: `<img>`, `<br>`, `<hr>`.

The Document Skeleton

```
1 <!doctype html>
2 <html lang="vi">
3   <head>
4     <meta charset="utf-8">
5     <meta name="viewport" content="width=device-width, initial-scale=1">
6     <title>Page Title</title>
7   </head>
8   <body>
9     <!-- Visible content goes here -->
10  </body>
11 </html>
```

- lang="vi" helps screen readers & browsers understand the page language.
- meta charset="utf-8": displays Vietnamese text correctly.
- meta viewport: correct mobile rendering (needed for Week 3 responsive design).



Basic Content Tags

- **Headings:** `<h1>` ... `<h6>` – by **meaning/hierarchy**, never skip a level.
- **Paragraph:** `<p>`; line break `<br>`; horizontal rule `<hr>`.
- **Emphasis:** `<strong>` (importance), `<em>` (stress/emphasis).
- Tip: choose a tag by its **content role**, not by how it happens to look.

Links, Images, and Lists

```
1 <a href="https://example.com">External link</a>
2 <a href="#contact">In-page link</a>
3
4 
5
6 <ul>                                <!-- unordered list -->
7   <li>Item A</li>
8   <li>Item B</li>
9 </ul>
10 <ol> ... </ol>                    <!-- ordered list -->
```

- href: link destination (absolute/relative URLs, from Week 1).
- alt: required for meaningful images (accessibility).



Grouping and Display Flow

- `<div>`: generic **block**-level grouping container.
- `<span>`: generic **inline** grouping container.
- **Block vs. inline:** block takes the full line (div, p, h1...); inline stays within the line (span, a, strong...).
- Only reach for div/span when **no meaningful tag fits** (see Part 2).



Further Reading (Part 1)

Optional – self-study handout

- Full list of HTML tags; meta tags for SEO.
- HTML entities: `&`, `<`, ` `...
- History & versions of HTML.
- Source: w3schools – *HTML Basic*, *HTML Elements*, *HTML Attributes*.



Lab 1: Profile / Intro Page

Hands-on practice

Build index.html with a full skeleton; inside `<body>` add:

- One `<h1>` heading, 1-2 `<p>` paragraphs.
- One `<a>` link, one `<img>` with alt.
- One `<ul>` list (skills / products / ...).
- *Finished early?* Add a table of contents using `<a>` links to an id on the page.
- *You may use your team's project topic instead of a personal profile.*



Semantic HTML



What Is Semantic HTML, and Why?

- Choose tags by **meaning**, not by appearance.
- **Accessibility**: assistive tools can understand page structure.
- **SEO**: search engines can read the role of your content.
- **Teamwork**: code communicates intent – easier to read & maintain.

Landmark Regions

```
1 <body>
2   <header> ... </header>    <!-- top of page: logo, title -->
3   <nav> ... </nav>          <!-- navigation bar -->
4   <main>                    <!-- the ONE main content area -->
5     <section> ... </section>
6     <article> ... </article>
7   </main>
8   <aside> ... </aside>      <!-- secondary content, sidebar -->
9   <footer> ... </footer>    <!-- bottom of page -->
10 </body>
```

Sketch a typical page layout on the whiteboard together.

div Soup vs. Semantic

Renders identically, but the semantic version **reads its own intent**:

```
1 <!-- Weak: role is unclear -->
2 <div class="top">...</div>
3 <div class="content">...</div>
4
5 <!-- Good: role is clear -->
6 <header>...</header>
7 <main>...</main>
```

Rule of thumb: only use div/span when **no semantic tag fits**.

Further Reading (Part 2)

Optional – self-study handout

- figure/figcaption, time, details/summary, mark.
- ARIA roles.
- Source: w3schools – *HTML Semantic Elements*.



Lab 2: Semantic-ize Your Page

Hands-on practice

- Wrap your Lab 1 content into landmark regions:
 - `<header>` (title), `<nav>` (a few menu links)
 - `<main>` → `<section>` (content), `<footer>` (closing info)
- Open **DevTools** to verify the landmark tree is correct.



Tables



When to Use a Table

- Use tables for **genuinely tabular data** (rows $\times$ columns).
- **Do not** use tables for **page layout** – that is CSS's job.
- DS/AI examples: metric comparisons, experiment logs, confusion matrices.

Core Tags and Table Structure

```
1 <table>
2   <caption>Model performance comparison</caption>
3   <thead>
4     <tr><th>Model</th><th>Accuracy</th><th>F1</th></tr>
5   </thead>
6   <tbody>
7     <tr><td>Logistic Regression</td><td>0.91</td><td>0.88</td></tr>
8     <tr><td>Random Forest</td><td>0.94</td><td>0.91</td></tr>
9   </tbody>
10 </table>
```

- Core: table, tr (row), th (header cell), td (data cell).
- Structure: caption, thead, tbody, tfoot.
- Use th for headers to **preserve meaning & accessibility**.

Merging Cells: colspan & rowspan

- colspan="n": a cell spans **n columns**.
- rowspan="n": a cell spans **n rows**.
- Use when grouping columns by dataset / stage / model family.

```
1 <th colspan="2">Dataset A</th>
```

Further Reading (Part 3)

Optional – self-study handout

- The scope attribute; headers/id links for complex tables.
- Styling & coloring tables with CSS (Week 3).
- Source: w3schools – *HTML Tables*.



Lab 3: Data Table

Hands-on practice

Build a table with:

- `<caption>`, `<thead>` (headers as `<th>`), `<tbody>` (≥ 3 rows).
- **At least one** `colspan` to group columns.
- Example: a model-comparison table, **or** your team project's data table.
- Add the table inside a new `<section>` within `<main>`.



Forms & Data Submission



Why Forms Matter, and the `<form>` Tag

- A form is the **data entry point** – how users send data to a server.
- This is exactly what connects to your backend (FastAPI) later on.
- Two vital attributes of `<form>`:
 - action: **where** it's sent (the endpoint).
 - method: **how** it's sent (GET / POST).

Common Form Controls

```
1 <label for="ten">Full name</label>
2 <input id="ten" name="ten" type="text" required>
3
4 <input name="tuoi" type="number">
5 <input name="email" type="email">
6 <input name="gioitinh" type="radio" value="nam"> Male
7
8 <select name="thanhpho">
9   <option value="hn">Hanoi</option>
10 </select>
11
12 <textarea name="ghichu"></textarea>
13 <button type="submit">Submit</button>
```

- input has many types: text, number, email, password, radio, checkbox, date, file.
- label binds to an input via for/id – clicking it focuses the input; screen readers announce it.



The `name` **Attribute** – The Most Important One

- `name` is the **key** in the key-value pair sent to the server.
- The backend (FastAPI) reads submitted data **by** `name`.
- **No `name` = the data is never sent.**
- Example: `name="bedrooms"` → server-side parameter `bedrooms`.

GET **vs.** POST

	GET	POST
Data travels in	URL (query string)	Request body
Best for	search / filtering	sensitive / large data
Bookmarkable, shareable	Yes	No
Typical example	search results page	login / create / upload

Live demo: same form – method="get" changes the **URL**; method="post" leaves the URL unchanged, data travels hidden.



Client-Side Validation

- required, type="email", min / max, pattern.
- Convenient for users, but **not trustworthy on its own**.
- Recall Week 1: **the server must always re-validate** – the frontend runs on the user's machine, and anyone can bypass it.

A Complete Form Example

```
1 <form action="/predict" method="post">
2   <label for="address">Address</label>
3   <input id="address" name="address" type="text" required>
4
5   <label for="bedrooms">Bedrooms</label>
6   <select id="bedrooms" name="bedrooms">
7     <option value="1">1</option>
8     <option value="2">2</option>
9   </select>
10
11   <button type="submit">Predict</button>
12 </form>
```


Further Reading (Part 4)

Optional – self-study handout

- fieldset/legend, placeholder, autocomplete, maxlength.
- Every input type; accept for file uploads.
- Advanced client-side validation.
- Source: w3schools – *HTML Forms, HTML Input Types*.

Lab 4: Data-Entry Form

Hands-on practice

Build a form (**every control must have a** name):

- One input type="text" (with <label>), one input type="number".
- One <select>, one radio group, one submit button.
- Set action="/predict" (or your team project's endpoint).
- **Experiment:** switch method="get" → submit → inspect the URL; switch to method="post" → compare.
- Add required to a few fields; try submitting them empty.



Accessibility Basics



5 Accessibility Habits

1. lang on the `<html>` tag (already done in the skeleton).
2. alt on every meaningful image.
3. label for every input (for/id).
4. Sensible heading hierarchy ($h1 \rightarrow h2 \rightarrow \dots$, never skip a level).
5. Sufficient text/background color contrast.



Why Accessibility Matters

- Serves screen-reader users and many other kinds of users.
- Data dashboards typically serve a wide audience – accessibility is **engineering quality**, not decoration.
- Cheap if done from the start; expensive to retrofit later.

Self-Check Checklist (paste this onto your page)



- ☐ `<html lang="vi">`
- ☐ Every `<img>` has a descriptive alt
- ☐ Every input has a `<label>` bound via for/id
- ☐ Headings follow their hierarchy, no skipped levels
- ☐ Text/background contrast is sufficient

Further Reading (Part 5)

Optional – self-study handout

- ARIA roles; the WCAG standard.
- Keyboard navigation; focus management.
- Source: w3schools / MDN – *Accessibility*.



Summary and Next Session

- **HTML** is the structural skeleton: elements, attributes, and a nested tree.
- **Semantic tags** make pages accessible, SEO-friendly, and easy to maintain.
- **Tables** for structured data; **forms** for sending data to a server.
- You now have a working page skeleton for your project screens.
- Next: **CSS** styles this skeleton (Week 3); **JavaScript** brings it to life (Week 4).

Questions & Answers